

UNIVERSIDAD NACIONAL AGRARIA DE LA SELVA
FACULTAD DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS
ESCUELA PROFESIONAL DE INGENIERÍA EN CIBERSEGURIDAD



Título:

**ANÁLISIS E IMPLEMENTACIÓN DE MODELOS DE EJECUCIÓN
MULTIHILO EN C**

Asignatura:

ESTRUCTURA DE DATOS Y ALGORITMOS

Docente:

Mg. CARLOS ABRAHAM RIOS RIVERA

Estudiante:

CONDOR POMA, MIGUEL ANGEL

Tingo María – Perú

2026

RESUMEN

Este informe analiza los modelos de ejecución en sistemas computacionales. Los modelos son el secuencial, concurrente, paralelo, productor–consumidor, maestro–trabajador y de hilos independientes. Estos modelos son fundamentales para organizar y controlar la ejecución de tareas. Influyen en el rendimiento, la escalabilidad y el uso eficiente de los recursos computacionales.

El estudio combina teoría y práctica. Se describen los principios de cada modelo y se implementan ejemplos utilizando el lenguaje de programación C. Se eligió C porque permite trabajar cerca del hardware. Esto facilita la comprensión de la gestión de memoria, procesos, hilos y mecanismos de sincronización.

También se comparan los modelos. Se evalúan sus ventajas, limitaciones y escenarios de aplicación. Esto ayuda a identificar cuándo cada modelo es más adecuado. Se consideran factores como la complejidad del problema, la concurrencia requerida y la disponibilidad de recursos.

En resumen, este informe ayuda a mejorar las competencias en el diseño y análisis de algoritmos eficientes. También ayuda a entender la interacción entre software y arquitectura del sistema. Estos son aspectos clave para un ingeniero en computación.

INDICE

I. INTRODUCCIÓN.....	4
1.1. Objetivos:.....	5
II. CONTENIDO DE INVESTIGACION.....	6
2. Ejecución multihilos.....	6
2.1 Modelo secuencial.....	7
2.2 Modelo concurrente.....	9
2.3 Modelo paralelo.....	11
2.4 Modelo Productor-Consumidor.....	13
2.5 Modelo Maestro-Trabajador.....	15
2.6 Modelo hilos independientes.....	17
III. DISCUCIONES.....	19
IV. CONCLUSIONES.....	21
V. RECOMENDACIONES.....	22
VI. REFERENCIAS BIBLIOGRAFICAS.....	23
VII. ANEXOS.....	24

I. INTRODUCCIÓN

En el desarrollo de sistemas computacionales modernos, la forma en que se ejecutan los programas es clave para el rendimiento, la eficiencia y la escalabilidad de las aplicaciones. Los modelos de ejecución son abstracciones que definen cómo se organizan, planifican y procesan las tareas dentro de un sistema. Permiten gestionar de manera adecuada los recursos disponibles como la CPU, la memoria y los dispositivos de entrada/salida.

En el estudio de la estructura de datos y algoritmos, estos modelos son muy importantes. Influyen en la forma en que se resuelven los problemas y en el tiempo de ejecución y el uso eficiente de los recursos.

Tradicionalmente, el modelo secuencial ha sido la base de la programación. Se caracteriza por la ejecución lineal de instrucciones. Sin embargo, con la evolución de la arquitectura de los sistemas hacia entornos multinúcleo y distribuidos, se ha hecho necesario el uso de modelos más avanzados como la concurrencia y el paralelismo. Estos permiten la ejecución simultánea de múltiples tareas.

La concurrencia y el paralelismo presentan nuevos desafíos. Por ejemplo, la necesidad de sincronización, la prevención de condiciones de carrera y la correcta comunicación entre procesos o hilos.

En este contexto, surgen patrones de diseño como productor–consumidor y maestro–trabajador. Facilitan la estructuración de soluciones eficientes en problemas donde existe interacción entre múltiples tareas.

El uso de hilos independientes permite maximizar el aprovechamiento del paralelismo disponible. Se pueden ejecutar tareas sin dependencia directa entre ellas.

En este informe, se realiza un análisis detallado de estos modelos de ejecución. Se acompaña de su implementación práctica utilizando el lenguaje de programación C.

La elección de C permite trabajar a un nivel cercano al hardware. Facilita la comprensión de aspectos fundamentales como la gestión de memoria, la creación y sincronización de hilos, y la interacción directa con el sistema operativo.

De esta manera, se busca no solo comprender los fundamentos teóricos. También se busca evidenciar su aplicación práctica en el desarrollo de soluciones eficientes dentro del ámbito de la computación.

1.1. Objetivos:

- Analizar los diferentes modelos de ejecución en sistemas computacionales, comprendiendo sus principios de funcionamiento y su impacto en el rendimiento de los algoritmos.
- Implementar los principales modelos de ejecución utilizando el lenguaje de programación C, con el fin de evaluar su comportamiento y eficiencia en distintos escenarios.

II. CONTENIDO DE INVESTIGACION

2. Ejecución multihilos

En los sistemas computacionales modernos, la ejecución multihilos es clave para ejecutar tareas de forma concurrente y paralela. Un hilo es la unidad más pequeña de ejecución dentro de un proceso. Comparte recursos como memoria, archivos y estado del sistema con otros hilos del mismo proceso.

Usar varios hilos permite dividir un programa en unidades de ejecución que pueden ejecutarse al mismo tiempo. Esto depende de la arquitectura del procesador. En sistemas multinúcleo, el sistema operativo puede asignar hilos a diferentes núcleos para ejecutarlos simultáneamente.

Sin embargo, los hilos comparten el mismo espacio de memoria. Esto introduce desafíos como la sincronización y coordinación del acceso a recursos compartidos. Problemas como condiciones de carrera, datos inconsistentes y bloqueos pueden ocurrir si no se controlan adecuadamente. Para evitar esto, se usan mecanismos como mutex, semáforos o variables de condición.

De este enfoque multihilo se derivan varios modelos de ejecución. Cada uno tiene un propósito específico. El modelo concurrente se enfoca en la ejecución intercalada de tareas. El modelo paralelo se enfoca en la ejecución simultánea de subprocesos independientes. Patrones como productor-consumidor o maestro-trabajador organizan la comunicación y distribución del trabajo entre hilos.

En este contexto, los modelos de ejecución analizados pueden entenderse como formas de gestionar y coordinar hilos dentro de un sistema. El objetivo es optimizar el uso de recursos y mejorar el rendimiento computacional.

2.1 Modelo secuencial

a) Definición

El modelo de ejecución secuencial es un enfoque donde las instrucciones de un programa se ejecutan una después de otra, en orden. Sigue exactamente lo que el código dice. En este modelo, cada cosa se hace antes de que comience la siguiente. Solo hay un hilo de ejecución.

Este tipo de ejecución no permite hacer varias cosas al mismo tiempo. No hay concurrencia ni paralelismo. Por eso, es fácil controlar el flujo y predecir lo que pasará. Esto hace que sea más fácil implementar y analizar los algoritmos.

b) Características

- Ejecución paso a paso (entrada → procesamiento → salida).
- Uso de un solo hilo de ejecución.
- No existe concurrencia.
- Flujo de control determinista.
- No requiere mecanismos de sincronización.

c) Ventajas y desventajas

Ventajas

- Simplicidad en el diseño del programa.
- Facilidad para depuración.
- Bajo uso de recursos del sistema.

Desventajas

- No aprovecha múltiples núcleos del procesador.
- Mayor tiempo de ejecución en problemas grandes.
- Baja eficiencia en tareas que podrían ejecutarse en paralelo.

d) Caso de uso

El modelo secuencial es adecuado para aplicaciones simples o donde las operaciones dependen directamente de resultados previos. Es común en programas educativos, procesamiento básico de datos y algoritmos donde no es necesaria la ejecución simultánea de tareas.

e) Implementación en C

La implementación completa del modelo secuencial se encuentra disponible en el repositorio GitHub del proyecto.

A continuación, se muestra un fragmento representativo del código:

```
int main() {
    int dato;
    int resultado;

    printf("=== MODELO SECUENCIAL ===\n");

    // Paso 1: Entrada
    dato = leer_datos();

    // Paso 2: Procesamiento
    resultado = procesar_datos(dato);

    // Paso 3: Salida
    mostrar_resultado(resultado);

    printf("Fin del programa\n");

    return EXIT_SUCCESS;
}
```

f) Análisis del código

Este fragmento evidencia el flujo lineal característico del modelo secuencial, donde cada etapa del programa (entrada, procesamiento y salida) se ejecuta en un orden estrictamente definido. No existe concurrencia ni paralelismo, ya que cada operación depende de la finalización de la anterior.

2.2 Modelo concurrente

a) Definición

El modelo de ejecución concurrente permite que varias tareas se ejecuten de manera intercalada en un mismo periodo de tiempo. Esto no significa que se ejecuten al mismo tiempo. En su lugar, se basa en la existencia de varios hilos o procesos que comparten recursos del sistema. Esto requiere mecanismos de sincronización.

A diferencia del modelo secuencial, la concurrencia mejora la eficiencia del sistema. Esto se logra al gestionar varias tareas de forma organizada. Sin embargo, al compartir recursos, pueden surgir problemas. Algunos de estos problemas son condiciones de carrera, inconsistencias en los datos y conflictos en la ejecución.

Para evitar estos problemas, se utilizan mecanismos como los mutex. Los mutex permiten controlar el acceso a secciones críticas del código. De esta manera, se garantiza la integridad de los datos compartidos.

b) Características

- Ejecución intercalada de múltiples tareas.
- Uso de múltiples hilos o procesos.
- Existencia de recursos compartidos.
- Necesidad de sincronización.
- Posibilidad de condiciones de carrera.

c) Ventajas y desventajas

Ventajas

- Mejora el aprovechamiento del tiempo de CPU.
- Permite ejecutar múltiples tareas de manera eficiente.
- Mayor capacidad de respuesta en aplicaciones.

Desventajas

- Mayor complejidad en el diseño.
- Riesgo de condiciones de carrera.
- Necesidad de mecanismos de sincronización (mutex, semáforos, etc.).

d) Caso de uso

El modelo concurrente es ampliamente utilizado en aplicaciones donde múltiples tareas deben ejecutarse de manera independiente pero coordinada, como en servidores web, sistemas operativos, procesamiento de solicitudes y aplicaciones que requieren interacción continua con el usuario.

e) Implementación en C

La implementación completa del modelo concurrente se encuentra disponible en el repositorio GitHub del proyecto.

A continuación, se muestra un fragmento representativo del código:

```
for (int i = 0; i < NUM_HILOS; i++) {  
    pthread_create(&hilos[i], NULL, incrementar_contador, NULL);  
}  
  
for (int i = 0; i < NUM_HILOS; i++) {  
    pthread_join(hilos[i], NULL);  
}
```

f) Análisis del código

Este fragmento evidencia la creación y ejecución concurrente de múltiples hilos, los cuales trabajan sobre una tarea común. Cada hilo ejecuta la función `incrementar_contador`, accediendo a una variable compartida protegida mediante un mutex. La sincronización mediante `pthread_join` garantiza que el programa principal espere la finalización de todos los hilos antes de continuar.

2.3 Modelo paralelo

a) Definición

El modelo de ejecución paralelo se basa en hacer varias tareas al mismo tiempo. Esto se hace para resolver un problema de manera más rápida. En lugar de hacer una tarea y luego otra, se hacen varias tareas a la vez. Esto se puede hacer porque los procesadores tienen varios núcleos que pueden trabajar al mismo tiempo.

Este modelo es útil porque reduce el tiempo que se necesita para resolver un problema. Esto se puede hacer si el problema se puede dividir en partes que no dependen una de la otra. Entonces, se puede asignar cada parte a un hilo o proceso diferente. De esta manera, se puede trabajar en todas las partes al mismo tiempo.

El paralelismo se utiliza en aplicaciones que necesitan mucha velocidad y eficiencia. Por ejemplo, se utiliza en el procesamiento de grandes cantidades de datos y en algoritmos que necesitan ser lo más rápidos posible. Esto se debe a que el paralelismo puede reducir significativamente el tiempo que se necesita para completar una tarea.

b) Características

- Ejecución simultánea de múltiples tareas.
- Uso de múltiples núcleos del procesador.
- División del problema en subproblemas independientes.
- Necesidad de sincronización al final (reducción de resultados).
- Mayor aprovechamiento de recursos del sistema.

c) Ventajas y desventajas

Ventajas

- Reducción del tiempo de ejecución.
- Mejor aprovechamiento del hardware.
- Escalabilidad en sistemas multinúcleo.

Desventajas

- Mayor complejidad en el diseño.
- Posible sobrecarga en la creación de hilos.
- Requiere dividir correctamente el problema.

d) Caso de uso

El modelo paralelo es adecuado para problemas que pueden dividirse en partes independientes, como el procesamiento de arreglos, operaciones matemáticas masivas, procesamiento de imágenes, simulaciones y análisis de grandes volúmenes de datos.

e) Implementación en C

```
for (int i = 0; i < NUM_HILOS; i++) {  
    datos[i].id = I;  
    datos[i].inicio = i * tam_bloque;  
    datos[i].fin = (i == NUM_HILOS - 1) ? TAM : (i + 1) * tam_bloque;  
    pthread_create(&hilos[i], NULL, suma_paralela, &datos[i]);  
}
```

f) Análisis del código

Este fragmento evidencia la división del problema en múltiples subprocesos, donde cada hilo recibe una porción específica del arreglo para su procesamiento. Esta distribución del trabajo permite que las tareas se ejecuten de manera simultánea, aprovechando el paralelismo del sistema. Posteriormente, los resultados parciales son combinados para obtener el resultado final.

2.4 Modelo Productor-Consumidor

a) Definición

El modelo productor-consumidor es un patrón muy común para organizar sistemas que trabajan al mismo tiempo. En este modelo, algunos procesos o hilos crean datos y los almacenan en un lugar compartido. Otros procesos o hilos se encargan de procesar esos datos.

Para que esto funcione bien, necesitamos encontrar formas de coordinar el acceso a los datos compartidos. Si no lo hacemos, podemos tener problemas como sobrescribir datos o intentar acceder a lugares vacíos. Para evitar esto, se utilizan herramientas como semáforos y mutex. Estas herramientas nos permiten controlar cuándo hay espacio disponible en el buffer y asegurarnos de que solo un proceso pueda acceder a los datos en un momento dado.

El modelo productor-consumidor es muy importante en aplicaciones donde hay una relación entre la creación y el uso de datos. Garantiza que la información fluya de manera controlada y eficiente.

b) Características

- Existencia de productores y consumidores.
- Uso de un buffer compartido.
- Sincronización mediante semáforos.
- Protección de la sección crítica con mutex.
- Coordinación entre hilos para evitar desbordamiento o subflujo.

c) Ventajas y desventajas

Ventajas

- Permite desacoplar la producción y el consumo de datos.
- Mejora la eficiencia en sistemas concurrentes.
- Controla el acceso a recursos compartidos de forma segura.

Desventajas

- Mayor complejidad en la implementación.
- Requiere manejo correcto de sincronización.
- Posible bloqueo si no se gestionan bien los semáforos.

d) Caso de uso

El modelo productor–consumidor es ampliamente utilizado en sistemas donde existe intercambio de datos entre procesos, como en buffers de entrada/salida, procesamiento de datos en tiempo real, sistemas de mensajería, y aplicaciones donde una tarea genera información que otra debe procesar.

e) Implementación en C

```
sem_wait(&empty);  
pthread_mutex_lock(&mutex);  
  
buffer[in] = item;  
in = (in + 1) % TAM_BUFFER;  
  
pthread_mutex_unlock(&mutex);  
sem_post(&full);
```

f) Análisis del código

Este fragmento evidencia el funcionamiento del productor al insertar elementos en un buffer compartido. Se utilizan semáforos para controlar la disponibilidad de espacio (`empty`) y elementos (`full`), mientras que el `mutex` garantiza la exclusión mutua en la sección crítica. Este mecanismo permite la correcta sincronización entre el productor y el consumidor, evitando condiciones de carrera y desbordamiento del buffer.

2.5 Modelo Maestro-Trabajador

a) Definición

El modelo maestro-trabajador es una forma de trabajar juntos muchos procesos. Aquí, un proceso llamado maestro se encarga de dar tareas a un grupo de trabajadores. Cada trabajador hace su tarea de manera independiente y luego informa sobre los resultados o para cuando no hay más tareas.

Este modelo funciona porque las tareas se dividen dinámicamente. Es decir, no se asignan tareas fijas a cada trabajador desde el principio. En lugar de eso, los trabajadores piden tareas cuando están listos. De esta manera, el trabajo se distribuye mejor y se utilizan los recursos del sistema de forma más eficiente.

El gran desafío de este modelo es asegurarse de que los trabajadores no se confundan cuando piden tareas. El maestro debe asegurarse de que cada trabajador reciba la tarea correcta y de que no haya problemas cuando varios trabajadores intentan acceder a la lista de tareas al mismo tiempo.

b) Características

- Un hilo principal actúa como maestro.
- Varios hilos actúan como trabajadores.
- Asignación dinámica de tareas.
- Uso de recursos compartidos.
- Necesidad de sincronización (mutex).

c) Ventajas y desventajas

Ventajas

- Balanceo dinámico de carga.
- Mejor aprovechamiento de los trabajadores.
- Escalable en número de hilos.

Desventajas

- Dependencia del punto central (maestro).
- Posible cuello de botella en acceso a tareas.
- Requiere sincronización para evitar inconsistencias.

d) Caso de uso

El modelo maestro–trabajador es utilizado en sistemas donde las tareas pueden ser divididas en unidades independientes, como procesamiento de lotes, cálculos paralelos, renderizado de gráficos, y sistemas de distribución de trabajo en servidores.

e) Implementación en C

```
int obtener_tarea() {
    int tarea = -1;

    pthread_mutex_lock(&mutex);

    if (indice_tarea < NUM_TAREAS) {
        tarea = tareas[indice_tarea];
        indice_tarea++;
    }

    pthread_mutex_unlock(&mutex);

    return tarea;
}
```

f) Análisis del código

Este fragmento evidencia el mecanismo de asignación dinámica de tareas, donde los trabajadores solicitan trabajo al maestro a través de una estructura compartida. El uso de un mutex garantiza el acceso seguro al índice de tareas, evitando condiciones de carrera. Este enfoque permite una distribución eficiente de la carga, ya que cada hilo obtiene nuevas tareas conforme queda disponible.

2.6 Modelo hilos independientes

a) Definición

El modelo de hilos independientes se basa en ejecutar varios hilos al mismo tiempo. Cada hilo hace algo diferente y no necesita hablar con los demás. Esto significa que cada hilo trabaja solo y hace su tarea de manera autónoma. De esta forma, las tareas se pueden hacer al mismo tiempo.

A diferencia de otros modelos, como el de productor y consumidor o el de maestro y trabajador, aquí no hay intercambio de información entre los hilos. No necesitan coordinarse entre sí. Esto hace que sea más fácil de implementar y reduce la complejidad de tener que sincronizar todo.

Este modelo es útil cuando las tareas son completamente independientes. Pueden ejecutarse al mismo tiempo sin afectar el resultado de las otras tareas.

b) Características

- Hilos completamente independientes.
- No existe comunicación entre tareas.
- No requiere sincronización ni mutex.
- Ejecución concurrente de funciones separadas.
- Simplicidad en el diseño.

c) Ventajas y desventajas

Ventajas

- Implementación sencilla.
- No requiere mecanismos de sincronización complejos.
- Buen aprovechamiento del paralelismo básico.
- Bajo riesgo de errores por concurrencia.

Desventajas

- No permite coordinación entre tareas.
- Limitado a problemas independientes.
- No es adecuado para flujos de datos dependientes.

d) Caso de uso

El modelo de hilos independientes se utiliza en aplicaciones donde las tareas no comparten datos ni requieren coordinación, como la ejecución de procesos en segundo plano, simulaciones independientes, tareas de logging, o módulos de un sistema que operan de manera aislada.

e) Implementación en C

```
pthread_create(&h1, NULL, tarea_lectura, NULL);  
pthread_create(&h2, NULL, tarea_procesamiento, NULL);  
pthread_create(&h3, NULL, tarea_escritura, NULL);
```

```
pthread_join(h1, NULL);  
pthread_join(h2, NULL);  
pthread_join(h3, NULL);
```

f) Análisis del código

Este fragmento evidencia la creación de múltiples hilos que ejecutan tareas independientes sin compartir recursos ni requerir sincronización adicional. Cada hilo realiza una función específica de manera autónoma, lo que permite la ejecución concurrente sin necesidad de mecanismos como mutex o semáforos. La sincronización final mediante `pthread_join` asegura la correcta finalización de todas las tareas antes de terminar el programa.

III. DISCUCIONES

El análisis de los distintos modelos de ejecución nos permite ver diferencias importantes en cómo se organizan y gestionan las tareas dentro de un sistema computacional. Cada modelo tiene características propias que lo hacen adecuado para ciertos tipos de problemas. Esto depende del nivel de dependencia entre tareas, el uso de recursos compartidos y la necesidad de paralelismo.

El modelo secuencial es el más simple. Las instrucciones se ejecutan de manera lineal. Aunque es fácil de implementar y predecir, no aprovecha las capacidades de los sistemas modernos con múltiples núcleos. Esto limita su rendimiento en problemas grandes.

El modelo concurrente es diferente. Permite la ejecución intercalada de múltiples hilos que comparten recursos. Esto mejora la utilización del procesador. Sin embargo, introduce complejidad adicional. Se necesitan mecanismos de sincronización para evitar problemas.

El modelo paralelo va más allá. Distribuye el trabajo en tareas independientes que pueden ejecutarse al mismo tiempo en diferentes núcleos. Esto reduce el tiempo de ejecución. Sin embargo, el problema debe poder dividirse adecuadamente en subproblemas independientes.

El modelo productor-consumidor introduce una estructura de coordinación. Se basa en un buffer compartido. La sincronización es esencial para garantizar la integridad de los datos. Este modelo se usa en sistemas donde hay un flujo continuo de información entre procesos.

El modelo maestro-trabajador permite la asignación dinámica de tareas. Esto mejora el balance de carga entre hilos. Sin embargo, puede generar problemas si no se maneja correctamente la sincronización.

Finalmente, el modelo de hilos independientes es la forma más simple de concurrencia. Cada hilo ejecuta una tarea aislada sin interacción con los demás. Esto reduce la complejidad de implementación. Sin embargo, limita la capacidad de coordinación entre tareas.

En general, hay un equilibrio entre complejidad y eficiencia. A mayor nivel de paralelismo y coordinación, mayor es el rendimiento potencial. Sin embargo, también aumenta la complejidad del diseño y la necesidad de sincronización.

IV. CONCLUSIONES

Al desarrollar e implementar diferentes modelos de ejecución en lenguaje C, se ve que la forma de organizar la ejecución de tareas afecta directamente el rendimiento, la eficiencia y la escalabilidad de los sistemas computacionales.

El modelo secuencial es sencillo y fácil de implementar, pero tiene limitaciones importantes en el uso de los recursos del sistema, especialmente en arquitecturas multinúcleo. Por otro lado, los modelos concurrente y paralelo permiten mejorar el uso del procesador, aunque son más complejos porque necesitan sincronización y coordinación entre hilos o procesos.

Los patrones como productor-consumidor y maestro-trabajador son importantes para resolver problemas donde se intercambian o distribuyen tareas de manera dinámica. Estos patrones permiten estructurar de manera eficiente la comunicación y la asignación de trabajo entre hilos.

El modelo de hilos independientes es útil cuando las tareas no necesitan interactuar, lo que simplifica la implementación. Sin embargo, esto limita la capacidad de coordinación entre procesos.

En resumen, no hay un modelo de ejecución óptimo para todos los casos. La elección del modelo depende de la naturaleza del problema, del nivel de dependencia entre tareas y de los recursos disponibles en el sistema. La forma de organizar la ejecución de tareas es crucial para el rendimiento y la eficiencia de los sistemas computacionales.

V. RECOMENDACIONES

Se recomienda analizar con cuidado las características del problema antes de elegir un modelo de ejecución. No todos los enfoques son adecuados para cualquier tipo de aplicación. El modelo secuencial puede ser suficiente para problemas simples. Sin embargo, los modelos concurrente y paralelo son mejores para escenarios que requieren más rendimiento y aprovechamiento de recursos.

Es importante implementar mecanismos de sincronización de manera correcta cuando se trabaja con recursos compartidos. Esto es especialmente cierto para modelos concurrentes y de comunicación entre hilos, como productor-consumidor y maestro-trabajador. El objetivo es evitar condiciones de carrera e inconsistencias en los datos.

En el modelo paralelo, se sugiere hacer una partición correcta del problema. Esto garantiza una distribución equilibrada de la carga de trabajo entre los hilos. De esta manera, se minimizan los tiempos de espera y los posibles cuellos de botella.

Finalmente, se recomienda utilizar herramientas de depuración y análisis de concurrencia al desarrollar aplicaciones multihilo. También es importante estudiar a fondo la arquitectura del sistema. El rendimiento de estos modelos depende directamente del hardware subyacente.

VI. REFERENCIAS BIBLIOGRAFICAS

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*. Wiley.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems*. Pearson.
- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language*. Prentice Hall.

VII. ANEXOS

```
> ./secuencial
=== MODELO SECUENCIAL ===
Ingrese un numero: 15
Resultado: 225
Fin del programa
> ./concurrency
=== MODELO CONCURRENTE ===
Valor final del contador: 400000
> ./paralelo
=== MODELO PARALELO ===
Suma total: 1000
> ./productor-consumidor
=== PRODUCTOR-CONSUMIDOR ===
Productor produce: 0
Productor produce: 1
Productor produce: 2
Productor produce: 3
Productor produce: 4
Consumidor consume: 0
Consumidor consume: 1
Consumidor consume: 2
Consumidor consume: 3
Consumidor consume: 4
Productor produce: 5
Productor produce: 6
Productor produce: 7
Productor produce: 8
Productor produce: 9
Consumidor consume: 5
Consumidor consume: 6
Consumidor consume: 7
Consumidor consume: 8
Consumidor consume: 9
```

```
> ./maestro-trabajador
=== MODELO MAESTRO-TRABAJADOR ===
Trabajador 0 procesa 1 → 1
Trabajador 0 procesa 2 → 4
Trabajador 0 procesa 3 → 9
Trabajador 0 procesa 4 → 16
Trabajador 0 procesa 5 → 25
Trabajador 0 procesa 6 → 36
Trabajador 0 procesa 7 → 49
Trabajador 1 procesa 8 → 64
Maestro: todas las tareas fueron procesadas
> ./hilosINDEPENDIENTES
=== MODELO DE HILOS INDEPENDIENTES ===
[Hilo 1] Leyendo datos...
[Hilo 3] Escribiendo resultados...
[Hilo 2] Procesando datos...
[Hilo 1] Lectura completada
[Hilo 3] Escritura completada
[Hilo 2] Procesamiento completado
Todas las tareas independientes finalizaron
```

Repositorio en GitHub :

["https://github.com/SB-mike010001/Estructura de Datos Algoritmos 2026/tree/main/week_3"](https://github.com/SB-mike010001/Estructura de Datos Algoritmos 2026/tree/main/week_3)